



Guía Complementaria Ayudantía 2

Ciclos

6 de agosto de 2026

Ayudante: Sergio Cárcamo N. - sergio.carcamon@usm.cl
Departamento de Informática UTFSM

1. Resumen de la materia

1.1. Introducción a los ciclos (o bucles)

Hasta ahora, nuestros programas se han ejecutado de manera lineal: línea por línea, de arriba hacia abajo, tomando decisiones ocasionales con `if`, pero siempre avanzando hacia el final. Sin embargo, en la programación real es extremadamente común necesitar que una instrucción (o un bloque entero de código) se ejecute múltiples veces. A estas estructuras de repetición controlada las llamamos **ciclos** o **bucles**.

1.2. El ciclo `while` (Mientras)

El ciclo `while` nos permite repetir un bloque de código **mientras** una condición específica sea verdadera (`True`).

Puedes pensarlo como un `if` que se repite: evalúa una condición, y si es verdadera, ejecuta su código indentado. La gran diferencia es que, al terminar de ejecutar ese bloque, el `while` "vuelve arribaz" revisa la condición nuevamente. Si sigue siendo verdadera, repite el bloque una vez más, y así sucesivamente hasta que la condición se vuelva falsa.

Para que un ciclo `while` funcione correctamente y no cause problemas, generalmente debe tener **tres partes fundamentales**:

1. **Inicialización:** Una variable definida *antes* de comenzar el ciclo, la cual será evaluada en la condición.
2. **Condición:** La expresión lógica (usando `>`, `<`, `==`, etc.) que determina si el ciclo entra a dar una vuelta o si se detiene.
3. **Actualización:** Un cambio que ocurre *dentro* del bloque del ciclo que modifica la variable inicial. ¡Si olvidamos este paso, la condición nunca cambiará, creando un temido **ciclo infinito** que colgará nuestro programa!

```
# Ejemplo 1: Un contador simple
contador = 1 # 1. Inicialización de la variable

# 2. Condición: Mientras el contador sea menor o igual a 5
while contador <= 5:
    print("El contador va en:", contador)
```

```
# 3. Actualización: Aumentamos en 1 el contador en cada vuelta (Fundamental)
contador = contador + 1

# Esta línea está fuera del while (sin indentar), se ejecuta al terminar el
ciclo
print("¡El ciclo ha terminado!")
```

Uso clásico: Validación de datos

Una de las aplicaciones más útiles y comunes del ciclo `while` es obligar al usuario a ingresar un dato correcto antes de que el programa pueda continuar. Si el usuario se equivoca, el ciclo lo *atrapa* y le vuelve a pedir el dato *mientras* su respuesta siga siendo incorrecta o inválida.

```
# Ejemplo 2: Validación de una contraseña
print("Bienvenido al sistema secreto.")

# Inicialización: Pedimos la contraseña por primera vez
contrasena = input("Ingrese la contraseña: ")

# Condición: Mientras la contraseña NO sea "1234", el acceso es denegado
while contrasena != "1234":
    print("Error: Contraseña incorrecta. Intente nuevamente.")

# Actualización: Volvemos a pedir la contraseña para que el usuario tenga otra
oportunidad
contrasena = input("Ingrese la contraseña: ")

# Si llegamos a esta línea, significa que el while terminó (la contraseña fue
correcta)
print("¡Acceso concedido! Sistema desbloqueado.")
```

2. Ejercicios Propuestos

Ejercicio 1: [Título del Ejercicio]

[Descripción del problema que el estudiante debe resolver].

Ejemplo: Si el usuario ingresa [Input], el programa debe imprimir [Output].

Ejercicio 2: [Título del Ejercicio]

[Descripción del problema que el estudiante debe resolver].

Nota: [Alguna restricción o pista para el estudiante].

3. Soluciones Propuestas

Notar que las soluciones propuestas de cada ejercicio **no necesariamente** son las únicas o mejores. De la misma forma, en evaluaciones (certámenes, tareas) se evaluará la funcionalidad y el pensamiento lógico por sobre la implementación, por lo que se recomienda comentar para que el evaluador sepa qué es lo que se quiso decir.

Solución Ejercicio 1

Solución Esperada

[Explicación breve de la lógica utilizada para llegar a la solución].

```
# Código de la solución esperada  
print("Hola Mundo")
```

Solución Ejercicio 2

Solución Esperada

[Explicación breve de la lógica utilizada para llegar a la solución].

```
# Código de la solución esperada  
resultado = 2 + 2
```